

1.基本变换

变换矩阵的结构

平移部分

旋转部分

缩放部分

齐次坐标部分

变换矩阵的组合

 平移矩阵

 旋转矩阵

 四元数

 缩放矩阵

示例代码

从变换矩阵中提取平移、旋转和缩放信息，并应用到物体的Transform组件上：

变换顺序的重要性

常见的变换顺序

变换矩阵的乘法顺序

例子

为什么顺序重要

 缩放再旋转

 旋转再平移

具体变换的效果

为什么缩放通常先计算

总结

1在Unity中，变换矩阵（Transformation Matrix）是一个4x4的矩阵，用于描述物体在三维空间中的变换，包括位置（平移）、旋转和缩放。理解变换矩阵的各个部分对于在计算机图形学和游戏开发中进行高级操作非常重要。以下是变换矩阵的详细解释。

变换矩阵的结构

一个4x4的变换矩阵通常表示为：

$$\begin{pmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{pmatrix}$$

在Unity中，矩阵的每一部分通常有特定的含义：

平移部分

平移部分位于矩阵的第四列，用于表示物体的位置。

$$\begin{pmatrix} m_{03} \\ m_{13} \\ m_{23} \end{pmatrix}$$

这部分直接影响物体在空间中的位置。

旋转部分

旋转部分位于矩阵的左上3x3子矩阵，用于表示物体的旋转。

$$\begin{pmatrix} m_{00} & m_{01} & m_{02} \\ m_{10} & m_{11} & m_{12} \\ m_{20} & m_{21} & m_{22} \end{pmatrix}$$

这个3x3矩阵是一个正交矩阵，其行向量（或列向量）分别表示物体的局部坐标轴方向。

提取旋转矩阵

为了获得纯旋转矩阵，我们需要将缩放因子除去。具体操作是将每个列向量归一化，即：

- 归一化列向量 x: $[m_{00}, m_{10}, m_{20}]^T / \text{缩放因子 x}$
- 归一化列向量 y: $[m_{01}, m_{11}, m_{21}]^T / \text{缩放因子 y}$
- 归一化列向量 z: $[m_{02}, m_{12}, m_{22}]^T / \text{缩放因子 z}$

缩放部分

缩放部分实际上也是嵌在左上3x3子矩阵中，不过缩放因素直接乘在旋转矩阵的各行上。因此，缩放变换会影响旋转矩阵的值，但不会单独存在于矩阵的某个区域。

提取缩放因子

缩放因子是列向量的模长，即：

- 缩放因子 x: $\sqrt{m_{00}^2 + m_{10}^2 + m_{20}^2}$
- 缩放因子 y: $\sqrt{m_{01}^2 + m_{11}^2 + m_{21}^2}$
- 缩放因子 z: $\sqrt{m_{02}^2 + m_{12}^2 + m_{22}^2}$

齐次坐标部分

齐次坐标部分位于矩阵的最后一行，用于保持矩阵的齐次性。

$$(0 \quad 0 \quad 0 \quad 1)$$

这部分通常为 (0,0,0,1)，确保矩阵运算的正确性。

变换矩阵的组合

在实际应用中，变换矩阵通常是平移、旋转和缩放矩阵的组合。以下是分别表示平移、旋转和缩放的矩阵形式：

平移矩阵

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

其中，(Tx,Ty,Tz) 表示平移向量。

旋转矩阵

例如，绕x轴的旋转矩阵：

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

绕x轴旋转 θ 角度

绕y轴和z轴的旋转矩阵类似。

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

绕y轴旋转 θ 角度

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

绕z轴旋转 θ 角度

四元数

上述的矩阵给大家一个错觉，以为最终的是采用欧拉角的方式的旋转矩阵，其实unity内部是使用四元数去表示旋转

具体参考

此处为语雀内容卡片，点击链接查看：https://www.yuque.com/huluobu-9zyc8/gtfcln/tyqg27s1tl93i11d?view=doc_embed

四元数转旋转矩阵

给定四元数 $q = (w, x, y, z)$ ，其对应的旋转矩阵 R 表示为：

$$R = \begin{pmatrix} 1 - 2y^2 - 2z^2 & 2xy - 2zw & 2xz + 2yw \\ 2xy + 2zw & 1 - 2x^2 - 2z^2 & 2yz - 2xw \\ 2xz - 2yw & 2yz + 2xw & 1 - 2x^2 - 2y^2 \end{pmatrix}$$

旋转矩阵与列向量

一个3x3旋转矩阵的列向量分别表示对象局部坐标系的X、Y和Z轴方向：

$$\begin{pmatrix} m_{00} & m_{01} & m_{02} \\ m_{10} & m_{11} & m_{12} \\ m_{20} & m_{21} & m_{22} \end{pmatrix}$$

- ``column0`` ($[m_{00}, m_{10}, m_{20}]^T$): X轴方向向量
- ``column1`` ($[m_{01}, m_{11}, m_{21}]^T$): Y轴方向向量 (上向量)
- ``column2`` ($[m_{02}, m_{12}, m_{22}]^T$): Z轴方向向量 (前向量)

为什么使用 ``column1`` 和 ``column2``

``Quaternion.LookRotation`` 使用前向量和上向量来构建四元数：

- 前向量 (Z轴方向)：决定物体面朝的方向。
- 上向量 (Y轴方向)：帮助确定物体的旋转倾斜，保证物体的“顶部”朝向正确的方向。

缩放矩阵

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

其中，(Sx,Sy,Sz) 表示缩放因子。

示例代码

在Unity中，你可以使用Matrix4x4类来创建和操作变换矩阵。下面是一个示例，展示如何创建一个包含平移、旋转和缩放的变换矩阵：

```
1  using UnityEngine;
2
3  public class TransformationMatrixExample : MonoBehaviour
4  {
5      void Start()
6      {
7          // 创建平移矩阵
8          Vector3 translation = new Vector3(1, 2, 3);
9          Matrix4x4 translationMatrix = Matrix4x4.Translate(translation);
10
11         // 创建旋转矩阵（绕 Y 轴旋转 45 度）
12         Quaternion rotation = Quaternion.Euler(0, 45, 0);
13         Matrix4x4 rotationMatrix = Matrix4x4.Rotate(rotation);
14
15         // 创建缩放矩阵
16         Vector3 scale = new Vector3(2, 2, 2);
17         Matrix4x4 scaleMatrix = Matrix4x4.Scale(scale);
18
19         // 组合变换矩阵
20         Matrix4x4 transformationMatrix = translationMatrix * rotationMatrix * scaleMatrix;
21
22         // 输出变换矩阵
23         PrintMatrix(transformationMatrix);
24     }
25
26     // 打印矩阵的函数
27     void PrintMatrix(Matrix4x4 matrix)
28     {
29         Debug.Log("Matrix:");
30         for (int row = 0; row < 4; row++)
31         {
32             string rowString = "";
33             for (int col = 0; col < 4; col++)
34             {
35                 rowString += matrix[row, col].ToString("F2") + "\t";
36             }
37             Debug.Log(rowString);
38         }
39     }
40 }
41
```

在进行矩阵变换时，计算的顺序非常重要，因为矩阵乘法是非交换的。这意味着矩阵相乘的顺序会影响最终的结果。

从变换矩阵中提取平移、旋转和缩放信息，并应用到物体的Transform组件上：

```
1  using UnityEngine;
2
3  public class MatrixOrderExample : MonoBehaviour
4  {
5      public Transform character; // 要应用变换的物体
6
7      void Start()
8      {
9          // 创建缩放矩阵
10         Vector3 scale = new Vector3(2, 2, 2);
11         Matrix4x4 scaleMatrix = Matrix4x4.Scale(scale);
12
13         // 创建旋转矩阵（绕 Y 轴旋转 45 度）
14         Quaternion rotation = Quaternion.Euler(0, 45, 0);
15         Matrix4x4 rotationMatrix = Matrix4x4.Rotate(rotation);
16
17         // 创建平移矩阵
18         Vector3 translation = new Vector3(1, 2, 3);
19         Matrix4x4 translationMatrix = Matrix4x4.Translate(translation);
20
21         // 按顺序组合变换矩阵
22         Matrix4x4 transformationMatrix = translationMatrix * rotationMatrix
23         * scaleMatrix;
24
25         // 输出变换矩阵
26         PrintMatrix(transformationMatrix);
27
28         // 应用变换矩阵到物体
29         ApplyMatrixToTransform(transformationMatrix, character);
30     }
31
32     // 打印矩阵的函数
33     void PrintMatrix(Matrix4x4 matrix)
34     {
35         Debug.Log("Matrix:");
36         for (int row = 0; row < 4; row++)
37         {
38             string rowString = "";
39             for (int col = 0; col < 4; col++)
40             {
41                 rowString += matrix[row, col].ToString("F2") + "\t";
42             }
43             Debug.Log(rowString);
44         }
45     }
46 }
```



```

44     }
45
46     // 将Matrix4x4应用到Transform组件
47     void ApplyMatrixToTransform(Matrix4x4 matrix, Transform transform)
48     {
49         // 提取平移
50         Vector3 translation = matrix.GetColumn(3);
51
52         // 提取缩放
53         Vector3 scale = new Vector3(
54             matrix.GetColumn(0).magnitude,
55             matrix.GetColumn(1).magnitude,
56             matrix.GetColumn(2).magnitude
57         );
58
59         //1.简单取法
60         //Quaternion q= matrix.rotation;
61         //Debug.Log("q:"+q);
62
63
64         // 提取旋转矩阵并归一化
65         Vector3 right = matrix.GetColumn(0) / scale.x;
66         Vector3 up = matrix.GetColumn(1) / scale.y;
67         Vector3 forward = matrix.GetColumn(2) / scale.z;
68
69         //2.获取全部矩阵信息
70         Matrix4x4 rotationMatrix = new Matrix4x4();
71         rotationMatrix.SetColumn(0, right);
72         rotationMatrix.SetColumn(1, up);
73         rotationMatrix.SetColumn(2, forward);
74         rotationMatrix.SetColumn(3, new Vector4(0, 0, 0, 1));
75
76         // 将旋转矩阵转换为四元数
77         Quaternion rotation = rotationMatrix.rotation;
78
79         //第二种旋转方式
80         //Quaternion rotation = Quaternion.LookRotation(rotationMatrix.Get
Column(2), rotationMatrix.GetColumn(1));
81
82         // 应用到Transform组件
83         transform.localPosition = translation;
84         transform.localRotation = rotation;
85         transform.localScale = scale;
86     }
87 }

```

变换顺序的重要性

在3D计算机图形学中，通常涉及三种基本变换：平移、旋转和缩放。变换顺序的不同会导致物体在空间中的最终位置和形状发生不同的变化。

常见的变换顺序

1. 缩放 (Scale)
2. 旋转 (Rotate)
3. 平移 (Translate)

通常的做法是首先缩放物体，然后旋转它，最后平移它。具体顺序如下：

变换矩阵的乘法顺序

假设我们有以下变换：

- 缩放矩阵 S
- 旋转矩阵 R
- 平移矩阵 T

为了实现上述的顺序，我们需要按照从右到左的顺序进行矩阵乘法：

$$M = T \times R \times S$$

这意味着当我们应用变换时，最先应用缩放，其次是旋转，最后是平移。

例子

假设我们有一个物体，我们希望先缩放它，然后旋转它，最后平移它。以下是具体的代码示例：

```
1  using UnityEngine;
2
3  public class MatrixOrderExample : MonoBehaviour
4  {
5      void Start()
6      {
7          // 创建缩放矩阵
8          Vector3 scale = new Vector3(2, 2, 2);
9          Matrix4x4 scaleMatrix = Matrix4x4.Scale(scale);
10
11         // 创建旋转矩阵（绕 Y 轴旋转 45 度）
12         Quaternion rotation = Quaternion.Euler(0, 45, 0);
13         Matrix4x4 rotationMatrix = Matrix4x4.Rotate(rotation);
14
15         // 创建平移矩阵
16         Vector3 translation = new Vector3(1, 2, 3);
17         Matrix4x4 translationMatrix = Matrix4x4.Translate(translation);
18
19         // 按顺序组合变换矩阵
20         Matrix4x4 transformationMatrix = translationMatrix * rotationMatrix * scaleMatrix;
21
22         // 输出变换矩阵
23         PrintMatrix(transformationMatrix);
24     }
25
26     // 打印矩阵的函数
27     void PrintMatrix(Matrix4x4 matrix)
28     {
29         Debug.Log("Matrix:");
30         for (int row = 0; row < 4; row++)
31         {
32             string rowString = "";
33             for (int col = 0; col < 4; col++)
34             {
35                 rowString += matrix[row, col].ToString("F2") + "\t";
36             }
37             Debug.Log(rowString);
38         }
39     }
40
41     // 将Matrix4x4应用到Transform组件
42     void ApplyMatrixToTransform(Matrix4x4 matrix, Transform transform)
43     {
```

```

44         // 提取平移
45         Vector3 translation = matrix.GetColumn(3);
46
47         // 提取旋转
48         Quaternion rotation = Quaternion.LookRotation(matrix.GetColumn
(2), matrix.GetColumn(1));
49
50         // 提取缩放
51         Vector3 scale = new Vector3(
52             matrix.GetColumn(0).magnitude,
53             matrix.GetColumn(1).magnitude,
54             matrix.GetColumn(2).magnitude
55         );
56
57         // 应用到Transform组件
58         transform.localPosition = translation;
59         transform.localRotation = rotation;
60         transform.localScale = scale;
61     }
62 }

```

为什么顺序重要

缩放再旋转

如果我们首先进行缩放，然后进行旋转，缩放会在物体的局部坐标系中发生，影响物体的形状，然后旋转会在缩放后的形状基础上进行。

旋转再平移

旋转会影响物体的方向和位置，而平移会将旋转后的物体移动到新的位置。

具体变换的效果

1. 缩放再旋转再平移：

- 物体首先按照局部坐标系进行缩放。
- 然后，物体绕其中心旋转。
- 最后，物体被平移到新的位置。

2. 平移再旋转再缩放：

- 物体首先被平移到新的位置。
- 然后，物体绕新的位置进行旋转。

- 最后，物体被缩放。

这种变换顺序通常用于场景中的物体需要在世界坐标系中进行旋转或缩放时。

为什么缩放通常先计算

1. **局部坐标系的缩放：** 缩放变换直接影响物体的尺寸。在物体的局部坐标系中进行缩放最为直观，因为这是物体最初的形状和尺寸。

如果在缩放之后再进行旋转和平移操作，缩放后的形状将被应用于旋转和平移，从而得到正确的变换效果。

2. **避免不一致性：** 如果先进行平移或旋转，然后再进行缩放，缩放将应用于整个变换后的坐标系，可能会导致不一致的效果。例如，先平移再缩放，会导致平移的距离也被缩放，改变了变换的预期效果。
3. **简化计算：** 先缩放再进行旋转和平移，可以简化变换矩阵的计算和理解。按照缩放、旋转、平移的顺序组合变换矩阵，更加直观和易于维护。

总结

在进行变换时，顺序非常重要。常见的顺序是先缩放，再旋转，最后平移。这是因为矩阵乘法是非交换的，改变顺序会导致最终结果不同。在实际应用中，必须根据具体需求选择合适的变换顺序，以达到预期的效果。